

CS246 Spring 2026 – Constructor Project

Plan of Attack (Due Date 1 Submission)

Team: Muneeb, Alex, Laith

Date Submitted: July 20, 2026

Due Date 2 Target: July 28, 2026, 5:00 pm

1. Overview

This document outlines our plan for implementing Constructor, our variant of Settlers of Catan for the University of Waterloo. Our finalized UML class model is included separately as `uml.pdf`. This plan breaks the remaining work into milestones with target completion dates, assigns responsibility for each component among the three of us, and answers the required design questions from the project specification.

Our design centers on five ideas we settled on during UML design:

- A Map composed of owned `Tile`, `Edge`, and `Vertice` objects, with `Edge/Vertice` holding non-owning pointers to each other and to their `Tile` neighbours to represent board adjacency.
- The Observer pattern (Subject/Observer) connecting Map to its Tiles, so a dice roll broadcasts to every tile rather than Map reaching into each tile's internals.
- The Strategy pattern applied twice: `BoardSource` (abstract) with `FileBoardSource/RandomBoardSource` concrete strategies for tile setup, and `Dice` (abstract) with `LoadedDice/FairDice` concrete strategies, swappable at runtime per player via `setDice()`.
- A single `Settlement` class with a `ResidenceLevel` field rather than a `Basement/House/Tower` inheritance hierarchy, since a residence's identity and owner must persist across upgrades and C++ objects cannot change their dynamic type after construction.
- Interface as a concrete controller class (MVC), holding the `vector<Player>`, the current Map, and the active `BoardSource`, and responsible for parsing commands and driving the game loop.

We are treating correctness and completeness as the top priority, per the guidelines' explicit statement that a working, well-documented partial implementation earns more than an ambitious but non-compiling one. Each milestone below ends with a compiling, runnable state.

2. Division of Responsibility

We split work by layer rather than by vertical feature slice, since our classes have relatively clean boundaries and this minimizes the number of files any two people need to touch simultaneously, reducing merge conflicts.

Muneeb — Board and Model core

`Tile`, `Edge`, `Vertice`, `Settlement`, `Observer`, `Subject`, `Map`,
`BoardSource/FileBoardSource/RandomBoardSource`, board display logic.

Alex — Player and game mechanics

Player, Dice/LoadedDice/FairDice, resource distribution logic, trading, geese movement and stealing, building-point tracking and win condition.

Laith — Interface, I/O, and persistence

Interface, command parsing and dispatch, save/load file I/O, command-line argument handling (-seed, -load, -board, -random-board), the help command, and end-to-end integration.

All three of us will contribute to testing throughout, and we will do a full integration pass together before Due Date 2 rather than leaving integration to one person at the end.

3. Milestones and Schedule

Monday, July 20 — Skeleton and compile baseline (all)

Set up the repository, module structure, and Makefile/order.txt. Stub out every class from the UML with empty method bodies so the project compiles from day one. Muneeb sets up Tile/Edge/Vertice/Map skeletons and hardcodes the board topology (adjacency between tiles, edges, and vertices) based on Figure 3, since the spec confirms this must be hardcoded or read from a fixed structure rather than derived by formula. Laith gets a minimal Interface::run() reading and echoing commands. Alex stubs Player and the Dice hierarchy. Target: project builds and runs, does nothing meaningful yet.

Tuesday, July 21 — Board setup and display

Muneeb implements FileBoardSource (parsing layout.txt) and Map::display() matching the exact ASCII format in Figures 1 through 3. Laith implements the beginning-of-game basement placement sequence (Blue, Red, Orange, Yellow, Yellow, Orange, Red, Blue) and wires it to Map. Alex implements Player's resource fields and the status print format. Target: a game can start, display an initial board with 8 basements placed, and print builder status.

Wednesday, July 22 — Building and roads

Muneeb finishes canBuildRoad/canBuildResidence adjacency checks on Map, and Edge::build/Vertice::buildSettlement/Vertice::improve. Alex wires Player::buildResidence/improveResidence/buildRoad to call into Map for legality and then Edge/Vertice for mutation, following the check-then-act contract we agreed on during UML review. Laith adds the build-road, build-res, and improve commands plus the resource-shortfall and illegal-build error messages. Target: full building phase works end to end for one player.

Thursday, July 23 — Dice, resource distribution, turn order

Alex implements LoadedDice/FairDice, Player::rollDice, and the Observer-driven resource distribution (Subject::notify to Tile::update to Vertice::giveMat to Player). Laith implements load/fair/roll and the four-player turn cycle. Muneeb supports by finalizing Tile::update's PARK and hasGeese skip logic. Target: multiple players can roll, receive resources, and pass turns correctly.

Friday, July 24 — Trading and geese

Alex implements `trade`, `loseHalfToGeese`, `stealFrom`, and the full geese-relocation flow (7 rolled, prompt for placement, list eligible victims, prompt for steal target, weighted-random resource transfer) per section 3.6. Muneeb supports with `Map::buildersOnTile` and `placeGoose`. Laith wires the `trade` command and geese prompts into the command loop. Target: geese and trading fully match the worked example in the spec (section 3.6).

Saturday, July 25 — Buffer and catch-up (all)

No new features scheduled. This day exists to absorb slippage from the prior four days, since building/resource/geese logic is the largest and riskiest chunk of the project. If we are on schedule, we begin the win condition and residences command.

Sunday, July 26 — Save/load, command-line arguments, win condition

Laith implements `save`, `-load`, `-board`, `-random-board`, and `-seed`, following the exact save-file format in section 3.7. Muneeb implements `RandomBoardSource`'s resource/value distribution. Alex implements the 10-point win condition, the "play again" prompt, and end-of-file handling (auto-save to `backup.sv`). Target: a saved game can be reloaded and continued identically.

Monday, July 27 — Full integration and testing (all)

All three of us test against each other's components. We run through the full command table (section 4.3), verify every printed message matches the spec's exact wording, and check boundary cases explicitly (building at the edge of the board, trading with no resources, geese landing back on the same tile, EOF mid-turn). We also begin drafting the Due Date 2 design document during any downtime.

Tuesday, July 28 — Freeze and submit

Per the guidelines' recommendation, we write no new code in the final six hours before the 5:00 pm deadline. This day is reserved for final compilation checks on a clean checkout, verifying `Makefile/order.txt/syslibs.txt`, and submitting to Marmoset with time to confirm it compiles there.

If we finish core requirements early, we will attempt the `no-delete/smart-pointer` bonus (largely already satisfied by our `unique_ptr` usage in `Vertice` and `Player`, but would require auditing every remaining raw owning pointer) and one or two extra features, selected once the core is stable.

4. Answers to Project Specification Questions

Question 1 (mandatory)

What design pattern could you use to choose between randomly setting up board resources and reading them from a file? Did you use it?

We use the Strategy pattern. `BoardSource` is an abstract class with a single method, `getTiles() : vector<TileInfo>`. `FileBoardSource` reads and parses `layout.txt` (or a file passed via `-board`), while `RandomBoardSource` generates a board following the distribution in section 3.5.5 (3 WFI, 3 HEAT, 4 BRICK, 4 ENERGY, 4 GLASS, 1 PARK, with the specified value distribution). `Interface` selects the appropriate concrete strategy based on command-line arguments and passes it to `Map`'s constructor, which calls `getTiles()` without needing to know which source it received. We did use this pattern, since it cleanly satisfies the "resilience to change" goal: adding a third board source

later (for example, a hexagonal layout for a future game mode, as referenced in Question 3) requires no changes to Map at all.

Question 2 (mandatory)

You must be able to switch between loaded and fair dice at run-time. What design pattern could you use to implement this feature? Did you use this design pattern? Why or why not?

We use Strategy again. Dice is an abstract class with a single `roll() : int` method. LoadedDice prompts the user with "Input a roll between 2 and 12:" and validates input; FairDice generates two random values from 1 to 6 and sums them. Each Player holds a `unique_ptr<Dice>` representing their current strategy, and `Player::setDice(unique_ptr<Dice>)` swaps it out in response to the load/fair commands. This keeps `Player::rollDice()` a single line, delegating entirely to whichever strategy is currently installed, and means the load/fair/roll commands never need to branch on dice type.

Question 3

We have defined the game of Constructor to have a specific board layout and size. Suppose we wanted to have different game modes (e.g. hexagonal tiles, a graphical display, different sized board for a different numbers of players). What design pattern would you consider using for all of these ideas?

We would extend the Strategy pattern we already use for BoardSource and generalize it further. A hexagonal-tile mode is really a different BoardSource implementation (or, if the topology itself changes shape, a different concrete Map built from a different adjacency description passed in at construction). A graphical display is a separate concern entirely from board generation, and we would address it by making Interface an abstract base with a `run()/display()`-style interface, and introducing a concrete GraphicalInterface alongside our current CLIInterface. This is the Strategy pattern applied to the View/Controller layer the same way we applied it to board setup. We chose not to implement this abstraction now, since the project only requires a single command-line interface and adding an unused abstraction layer would be premature generalization that increases complexity without a present benefit; however, our current design (with Interface cleanly separated from Map, and Map cleanly separated from its data source) means this extension would not require touching the Model layer at all if we did pursue it later.

Question 4

At the moment, all Constructor players are humans. However, it would be nice to be able to allow a human player to quit and be replaced by a computer player. If we wanted to ensure that all player types always followed a legal sequence of actions during their turn, what design pattern would you use? Explain your choice.

We would use the Template Method pattern (with Non-Virtual Interface, as covered in the course). We would introduce an abstract PlayerController with a single public, non-virtual `takeTurn()` method that encodes the fixed legal sequence: prompt for dice mode, roll, distribute resources, allow a bounded set of build/trade actions, then end the turn. Each step within `takeTurn()` that varies between a human and a computer (specifically, how a decision such as "which vertex to build on" gets made) would be a private pure virtual method, such as `chooseBuildTarget()`, overridden differently

by `HumanController` (reads from standard input) and `ComputerController` (runs a strategy). Since `takeTurn()` itself is non-virtual and not overridable, no subclass can skip a step or reorder the sequence, which is exactly the guarantee the question asks for. We considered this design but did not implement it, since our current specification only requires human players; introducing the `PlayerController` abstraction now would mean building and testing an interface with only one real implementation, which is not a good use of our limited time before Due Date 2.

Question 5

What design pattern would you use to allow the dynamic change of computer players, so that your game could support computer players that used different strategies, and were progressively more advanced/smarter/aggressive?

This builds directly on our answer to Question 4. Rather than growing a deeper `ComputerController` subclass hierarchy for every difficulty level (which would need recompilation and would multiply combinatorially if difficulty needed to vary independently along more than one axis, such as aggression versus risk tolerance), we would apply the Strategy pattern a second time, one layer beneath the Template Method structure from Question 4. `ComputerController` would hold a `unique_ptr<AIStrategy>`, an abstract class whose methods correspond to the same decision points `takeTurn()` already calls out to (`chooseBuildTarget()` and similar). Concrete strategies such as `EasyStrategy`, `MediumStrategy`, and `HardStrategy` would each implement those decision points differently, and swapping a computer player's difficulty at runtime would be a single `unique_ptr` reassignment, the same mechanism we already use for `Player::setDice`. We did not implement this, for the same reason as Question 4: computer players are outside the current specification, so building this abstraction now would mean maintaining an interface with no real caller.

Question 6

Suppose we wanted to add a feature to change the tiles' production once the game has begun. For example, being able to improve a tile so that multiple types of resources can be obtained from the tile, or reduce the quantity of resources produced by the tile over time. What design pattern(s) could you use to facilitate this ability?

We would use the Decorator pattern, the same pattern the course covers with the pizza topping example. A base `Tile`'s resource-production step would be wrapped by decorator objects implementing the same interface, such as `BonusResourceDecorator` (adds a second resource type to whatever the wrapped tile already produces) or `DiminishingYieldDecorator` (reduces the quantity produced on each successive trigger), each delegating to the object it wraps before or after applying its own modification. This avoids needing a dedicated subclass for every combination of modifiers a tile could accumulate over a game, the same way the pizza example avoids a subclass for every combination of toppings. We did not implement this, since our specification defines tile production as fixed for the entire game with no mechanism for players to alter it.

Question 7

Did you use any exceptions in your project? If so, where did you use them and why? If not, give an example of a place that it would make sense to use exceptions in your project and explain why you didn't use them.

We did not use exceptions as our primary error-handling mechanism, and made this choice deliberately rather than by omission. Nearly every failure case in the specification (illegal build location, insufficient resources, invalid command, invalid dice roll) is an expected, routine outcome of user input rather than an exceptional circumstance, and the spec asks for a specific printed message and continued execution in each case, not program termination or stack unwinding. Following the course's own guidance that error codes are appropriate when failure is a normal, anticipated outcome the caller is expected to check, we instead use `bool` return values (`Edge::build`, `Vertice::buildSettlement`, `Player::trade`, and so on) which `Interface` checks after every command and translates into the appropriate printed message.

The one place we did consider exceptions seriously is file I/O during `-load` and `-board`: if a specified file does not exist or is malformed, this is arguably a genuine exceptional circumstance rather than a routine game-rule violation, since the spec provides no defined message for it and we may assume well-formed input per the assignment's guarantees. We plan to let `ifstream`'s own failure state (`fail()`) surface this, checked once at startup in `Interface`, rather than introducing a custom exception type for a case the spec does not require us to handle gracefully. If we pursue the extra-credit challenge of a fully leak-free, exception-safe build, we will revisit this and ensure our `unique_ptr` usage throughout (`Vertice::house`, `Player::dice`, `Interface::boardSource`) already gives us the basic exception-safety guarantee for free, since RAII means any exception unwinding through these classes cannot leak the resources they own.

5. Risks and Mitigations

The largest risk is the building/adjacency logic (`canBuildRoad/canBuildResidence`), since it depends on the `Edge-Vertice-Tile` pointer topology being fully and correctly wired before any legality checking can be tested. We have scheduled this for Wednesday, immediately after board setup, and reserved Saturday as a dedicated buffer day specifically because we expect this to be where problems surface. The second risk is the geese/steal mechanic, since it is the most stateful and multi-step interaction in the spec; we have scheduled it with a full day and immediate testing against the worked example in section 3.6 rather than testing it only during final integration. Per the guidelines' advice, each of us will confirm the project still compiles and runs at the end of every work session, so we always have a submittable version regardless of what remains incomplete.